P. Areias simple C#

NUMERICAL & SCIENTIFIC COMPUTING EDITION

ORIENTATION — READ THIS FIRST

How to Use This Sheet

Read in this order; each step is self-contained and takes minutes, not hours.

1. **Value vs Reference** (next card) — the one model that, if wrong, produces bugs you cannot debug.
2. **From Fortran / C++ / Python** — translate what you already know.
3. **Program Structure → Control Flow → Arrays & Ranges**. You can now write a solver.
4. **Class, Records, Inheritance & Interfaces** — how to organise it.
5. **Testing** — before you optimise anything.
6. **Span → TensorPrimitives → Generic Math**. Only now think about speed.
7. Keep **Numerical Correctness Traps** (last page) at your elbow permanently.

Steps 1–5 are the language and its discipline. Everything after is *performance*, which is worth nothing until the answer is right.

MINIMUM VIABLE TOOLCHAIN

```
// $ dotnet new console -n Solver && cd Solver
// $ dotnet run                       // debug, slow, checked
// $ dotnet run -c Release            // ALWAYS benchmark this one
// $ dotnet new xunit -n Solver.Tests && dotnet test
// $ dotnet format                    // canonical layout
```

TRAP: Timing a Debug build is the most common benchmarking mistake. Measure with `-c Release`, or the number means nothing.

From Fortran / C++ / Python

What you write today	C#
`do i = 1, n`	`for (int i = 0; i < n; i++)` — 0-based, always
`a(i,j)`, column-major	`m[i,j]`, **row-major**; last index is contiguous
`real(8) :: x` / `double x`	`double x` — and `1.0` is already `double`
`1.0d0`	`1.0` (use `1.0f` to *force* `float`)
`allocate(a(n))`	`double[] a = new double[n];` — zero-filled
`deallocate` / `delete[]`	nothing: the GC owns managed memory
`module` / `#include`	`namespace` + `using` (names, not files)
`intent(in) :: x(:)`	`ReadOnlySpan<double> x` — no copy, no write
`pointer`, `T*`	a reference type; `T*` only inside `unsafe`
`nullptr` / `None`	`null`; the type `T?` says it may be
`size(a)` / `len(a)`	`a.Length`
`write(*,*) x` / `print(x)`	`Console.WriteLine($"{x:G17}")`
NumPy `a[2:8]` (a view)	`a.AsSpan(2, 6)`; `a[2..8]` **copies**
NumPy `a * b`	`TensorPrimitives.Multiply(a, b, y)`
`dot_product(a,b)`	`TensorPrimitives.Dot<double>(a, b)`
`matmul` / `a @ b`	not in the BCL — write it or take a dependency
free function `def f(x)`	must be a `static` method inside a type
`stop 1` / `exit(1)`	`return 1;` from `Main`

TRAP: Row-major versus column-major is the costliest porting mistake: a loop nest transcribed literally from Fortran strides the wrong way through cache. Iterate so the *last* index varies fastest.

Value vs Reference — Get This Right First

```
// class = REFERENCE type: the variable holds a reference to a
     heap object
public sealed class Node { public double X; }

var n1 = new Node { X = 1 };
var n2 = n1;  n2.X = 9;     // n1.X == 9   both names reach ONE
     object

// double, int, bool = VALUE types: the variable holds the data
     itself
double d1 = 1.0;
double d2 = d1;  d2 = 9.0; // d1 == 1.0   assignment COPIED the
     value

// Arrays are reference types, whatever they hold.
double[] a = new double[3];
double[] b = a;            b[0] = 7;  // a[0] == 7: alias, no copy
double[] c = a.AsSpan().ToArray();    // explicit copy (no LINQ)

// Arguments: the REFERENCE is passed by value.
static void Fill (double[] v) { v[0] = 1; }          // caller
     SEES this
static void Reset(double[] v) { v = new double[3]; } // caller
     sees NOTHING
```

DEFAULT EQUALITY IS NOT THE SAME

Kind	`a.Equals(b)` / `a == b` out of the box
`class`	reference identity — two equal meshes compare unequal
`record`	value equality, generated, `==` included
`double`, `int`	the value itself
`double[]`	reference identity — *not* the contents

WHICH DO I WRITE?

`sealed class` for anything with identity, ownership, or state — a mesh, a solver, a material model.

`record` when the object is defined entirely by its values and you want equality, `with`, and deconstruction generated for you. When unsure: `sealed class`.

TRAP: Two arrays with identical contents are *not* equal: `a == b` compares references, so it is `false` for distinct arrays and is almost never the test you wanted. Compare contents with `a.AsSpan().SequenceEqual(b)`.

Visibility, Lifetime & `static`

Modifier	Visible to
`public`	everyone, including other assemblies
`internal`	this assembly only — *default for top-level types*
`protected`	this type and anything deriving from it
`private`	this type only — *the default for members*
`protected internal`	this assembly *or* any derived type
`private protected`	derived types *within* this assembly
`file`	this source file only (source generators)

```
public sealed class Solver
{
  // const: compile-time literal, baked into every call site
  public const double Tol = 1e-10;

  // static readonly: one per type, set once, any expression
  public static readonly double Sqrt2 = Math.Sqrt(2.0);

  // readonly: one per instance, set in ctor or initializer only
  private readonly double[] _k;

  // static: belongs to the TYPE, not to any instance
  public static int InstanceCount;

  public Solver(int n) { _k = new double[n]; InstanceCount++; }
}
// A static class cannot be instantiated or inherited:
public static class Constants { public const double G = 9.80665; }
```

TRAP: Changing a `public const` in a library and recompiling only that library does *not* update consumers — the old literal is already inlined into their IL. For any value that might change, use `static readonly`.

Mutable `static` state is shared by every thread in the process. Either make it immutable, or protect it with a `lock`. Static fields are the usual reason a parallel run gives a different answer each time.

LANGUAGE FUNDAMENTALS

Program Structure & Namespaces

```
// File-scoped namespace is fine; keep the entry point explicit.
using System;
using System.Numerics;
using static System.Math;        // Sqrt(), Sin(), PI

namespace MyApp.Numerics;

using DoubleVector = double[];  // local type alias

public static class Program
```

```
{
    public static int Main(string[] args)
    {
        Console.WriteLine($"args = {args.Length}");
        double[] x = [1.0, 2.0, 3.0];
        Console.WriteLine($"norm = {Norm(x):G17}");
        return 0;
    }
    // Awaiting at the entry point? Change the signature, nothing
     else:
    //   public static async Task<int> Main(string[] args)
```

```
    // Tutorials usually drop Program/Main entirely ("top-level
     statements":
    // just write the statements in Program.cs). Identical
     semantics -- the
    // compiler synthesises this exact shape. Kept explicit here
     so the
    // structure of a program stays visible.

    private static double Norm(ReadOnlySpan<double> x)
    {
        double s = 0.0;
        for (int i = 0; i < x.Length; i++)
            s += x[i] * x[i];
        return Sqrt(s);
    }
}

internal sealed class InternalHelper { }
```

BUILD, RUN & PROJECT SWITCHES

```
// $ dotnet new console -n MyApp
// $ cd MyApp && dotnet run -c Release

// $ dotnet publish -c Release -r linux-x64 --self-contained

// .csproj essentials for strict numerical code
// <TargetFramework>net10.0</TargetFramework>
// <LangVersion>14.0</LangVersion>   // avoid latest for
     reproducibility
// <Nullable>enable</Nullable>
// <ImplicitUsings>enable</ImplicitUsings>
// <CheckForOverflowUnderflow>true</CheckForOverflowUnderflow>
// <AllowUnsafeBlocks>true</AllowUnsafeBlocks>  // only when
     needed
```

C# 14 Essentials

```
// Null-conditional assignment: RHS evaluated only if c != null
customer?.Order = GetCurrentOrder();
customer?.Count += 1;       // compound assignment is allowed
// customer?.Count++;       // not allowed

// nameof with an unbound generic type
string name = nameof(Dictionary<,>);    // "Dictionary"

// Extension members add instance-like members from static classes
// Use sparingly: prefer obvious APIs for numerical kernels.

// Field-backed properties: 'field' is contextual
public string Name {
  get;
  set => field = value ?? throw new ArgumentNullException(nameof(
     value));
}

// Partial constructors/events and user-defined compound
     assignment
// are mainly library/source-generator features; keep public APIs
     simple.
```

C# 14 is tied to the .NET 10 SDK in normal SDK-style projects. Prefer the target framework's default language version or pin `14.0`; avoid `latest` in reproducible builds.

Built-in Types Reference

Type	Notes
`bool`	`true` / `false`
`byte`, `sbyte`	8-bit unsigned / signed
`short`, `ushort`	16-bit
`int`, `uint`	32-bit; default for int literals
`long`, `ulong`	64-bit
`Int128`, `UInt128`	128-bit
`nint`, `nuint`	native size (32 on x86, 64 on x64)
`Half`	16-bit IEEE 754; `System.Half`
`float`	32-bit IEEE 754; suffix `f`
`double`	64-bit IEEE 754 – **default for FEM**
`decimal`	128-bit base-10; for money, *not* science
`Complex`	`System.Numerics.Complex`
`char`	UTF-16 code unit; `string`: immutable seq.

IEEE 754 ESSENTIALS

```
double inf = double.PositiveInfinity;
double nan = double.NaN;
double eps = double.Epsilon;           // smallest +denormal
const double Macheps = 2.220446049250313e-16; // nextafter(1)-1
const double UnitRoundoff = 1.1102230246251565e-16; // eps/2

bool ok = double.IsFinite(x);    // not NaN, not Infinity
bool sub = double.IsSubnormal(x);
```

TRAP: NaN breaks both equality and ordering. `x == double.NaN` is always `false` — test with `double.IsNaN(x)`. And `double.NaN.CompareTo(1.0)` returns -1 (NaN sorts as least) while `NaN < 1.0` is `false`: sorting and comparison disagree.

System.Math, double & Numeric Constants

Method	Notes
`Sin Cos Tan`	radians
`Asin Acos Atan`	inverses
`Atan2(y, x)`	full quadrant
`Sinh Cosh Tanh`	hyperbolic
`Sqrt Cbrt`	roots
`Pow(a, b)`	a^b
`Exp Log Log2 Log10`	exp & logs
`Abs Sign`	magnitude / ± 1
`Max Min Clamp`	$v \in [lo, hi]$
`Ceiling Floor Truncate`	rounding
`Round(x, n, mode)`	banker's by default
`FusedMultiplyAdd(a,b,c)`	$a \cdot b + c$ one rounding
`double.Hypot(x,y)`	$\sqrt{x^2 + y^2}$ overflow-safe
`ScaleB(x, n)`	$x \cdot 2^n$ exact
`ILogB(x)`	integer $\lfloor \log_2 \lvert x \rvert \rfloor$
`BitDecrement(x)`	next double toward $-\infty$
`BitIncrement(x)`	next double toward $+\infty$

CONSTANTS & PRECISION NOTES

```
double pi = Math.PI;      // 3.14159265358979...
double tau = Math.Tau;    // 2 * PI
```

`Math` is double; `MathF` is float. Static generic math members such as `double.Hypot` and `T.Hypot` complement `Math`. `FusedMultiplyAdd` gives one rounding instead of two; it may be faster when hardware and the JIT select fused instructions.

Operators & Pattern Matching

Expr	Meaning
`! || &&`	not / or / and (short-circuit)
`?? ??=`	null-coalescing / assign
`?. ?[]`	null-conditional access
`?.X = v`	null-cond. assign
`is T v`	type test + bind
`is not null`	non-null check
`is > 0`	relational pattern
`is (>0 and <N)`	combined patterns
`is [x, ..]`	list/slice pattern
`x as T`	safe cast; null on fail
`sizeof(T)`	bytes (unmanaged `T`)
`nameof(D<,>)`	string name; unbound
`checked{}`	overflow throws

```
// Switch expression: exhaustive, returns value
double abs = x switch {
  double.NaN => throw new ArithmeticException(), // test first
  >= 0       => x,
  < 0        => -x,
  _          => throw new ArithmeticException()
};
// Property pattern + tuple pattern
static int Quad((double X, double Y) p) => p switch {
  { X: > 0, Y: > 0 } => 1,   { X: < 0, Y: > 0 } => 2,
  { X: < 0, Y: < 0 } => 3,   { X: > 0, Y: < 0 } => 4,
  _                  => 0
};
```

Control Flow

```
if (x > 0) { } else if (x < 0) { } else { }
for (int i = 0; i < n; i++) { break; continue; }
foreach (var v in seq) { }       // cannot mutate seq
while (cond) { }
do { } while (cond);

// Switch statement: explicit break required
switch (kind) {
  case 1:          A(); break;
  case 2: case 3:  B(); break;
  case int v when v > 10:   C(v); break;
  default:         D(); break;
}
```

```
// Static local: no captures, no allocation
double Solve(double[] x) {
  static double Inv(double v) =>
      v == 0 ? throw new DivideByZeroException() : 1/v;
  return Inv(x[0]);
}
```

Exception Handling

```
try {
  Solve(K, rhs, x);
}
catch (FileNotFoundException ex) {
  Log($"missing: {ex.FileName}");
}
catch (ArithmeticException ex) when (ex.Message.Contains("nan")) {
  Recover();                    // exception filter
}
catch (Exception) {
  throw;                        // re-throw; keeps stack trace
                                // do NOT write 'throw ex;'
}
finally { /* always runs; release locks here */ }

// Guard helpers (CallerArgumentExpression makes good msgs)
ArgumentNullException.ThrowIfNull(coords);
ArgumentOutOfRangeException.ThrowIfNegative(n);
ArgumentOutOfRangeException.ThrowIfGreaterThan(i, max);

// Throw expression
var v = map.ContainsKey(k) ? map[k]
      : throw new KeyNotFoundException(nameof(k));
```

Exceptions are for exceptional paths. Validate at the public API boundary, avoid exception-driven control flow in kernels, and keep hot loops free of guards once preconditions have been checked.

Arrays, Ranges & Collection Expr.

```
double[]   a   = new double[n];       // zero-initialised
double[,]  mat = new double[r, c];    // 2-D rectangular
double[][] jag = new double[r][];     // jagged rows

// Index ^ and range ..
double   last = a[^1];         // a[a.Length - 1]
double[] mid  = a[2..^2];      // ALLOCATES (new array)
Span<double> view = a.AsSpan(2, n - 4); // does NOT allocate

// Bulk operations -- prefer over hand loops
Array.Copy(src, sOff, dst, dOff, count);
Array.Clear(a);                // zeroes every element
Array.Sort(keys, values);      // co-sort parallel arrays

// Collection expressions
int[]     p = [1, 2, 3];
List<int> q = [..p, 4, 5];     // spread operator
double[]  r = [..a, ..b];      // concatenate

// Named tuples
(double Lo, double Hi) MinMax(ReadOnlySpan<double> x) {
  double lo = x[0], hi = x[0];
  for (int i = 1; i < x.Length; i++) {
    if (x[i] < lo) lo = x[i];
    if (x[i] > hi) hi = x[i];
  }
  return (lo, hi);
}
var (lo, hi) = MinMax(data);    // deconstruct
```

TRAP: `a[2..8]` on an array allocates a new array. Inside loops, always use `a.AsSpan(2, 6)`.

TYPES & OBJECT-ORIENTED PROGRAMMING

Class — Modern Numerical Style

```
public sealed class Mesh(double[] coords) : IEquatable<Mesh>
{
  private readonly double[] _coords =
      coords ?? throw new ArgumentNullException(nameof(coords));

  public required string Name { get; init; }  // set by
     initializer
  public required int    Dim  { get; init; }

  public IReadOnlyList<double> Coords => _coords;

  // Field-backed property: 'field' is compiler-generated storage.
  public int NodeCount {
    get;
    set => field = value < 0
      ? throw new ArgumentOutOfRangeException(nameof(value))
      : value;
  } = coords.Length / 3;

  public bool Equals(Mesh? other) =>
      other is not null && Name == other.Name && Dim == other.Dim;

  public override bool Equals(object? obj) => obj is Mesh m &&
     Equals(m);
  public override int GetHashCode() => HashCode.Combine(Name, Dim)
     ;

  public static bool operator==(Mesh? a, Mesh? b) =>
      a is null ? b is null : a.Equals(b);
  public static bool operator!=(Mesh? a, Mesh? b) => !(a == b);
}

var mesh = new Mesh(coords) { Name = "beam", Dim = 3 };
```

Primary constructors place parameters in scope for instance initializers and members. They do not imply ownership: copy or validate mutable inputs when object invariants require it. Equal objects must hash equal, and hash values must remain stable while used as dictionary keys.

Records & Operator Overloading

```
// record class: reference type, value-based equality
// Compiler synthesises Equals, GetHashCode, ToString,
// operator==, copy ctor, Deconstruct, with-expressions.
public record Point(double X, double Y) {
  public double Norm => Math.Sqrt(X*X + Y*Y);
}
Point a = new(1, 2);
Point b = a with { Y = 5 };        // non-destructive copy
var (x, y) = a;                    // deconstruct

// A record carries operators like any other type. Seal it unless
// you mean to inherit: record equality compares the RUNTIME type,
// so a derived record never equals its base, even field-for-field
     .
public sealed record Vec3(double X, double Y, double Z)
{
  public double Norm  => Math.Sqrt(X*X + Y*Y + Z*Z);
  public Vec3   Unit  => this / Norm;

  // Operator overloading: must be 'static'
  // BOTH scalar orderings must be defined
  public static Vec3 operator+(Vec3 a, Vec3 b) =>
      new(a.X+b.X, a.Y+b.Y, a.Z+b.Z);
  public static Vec3 operator-(Vec3 a)              // unary
      => new(-a.X, -a.Y, -a.Z);
  public static Vec3 operator*(double s, Vec3 v) =>
      new(s*v.X, s*v.Y, s*v.Z);
  public static Vec3 operator*(Vec3 v, double s) => s * v;
  public static Vec3 operator/(Vec3 v, double s) =>
      new(v.X/s, v.Y/s, v.Z/s);

  public static double Dot(Vec3 a, Vec3 b) =>
      a.X*b.X + a.Y*b.Y + a.Z*b.Z;
  public static Vec3 Cross(Vec3 a, Vec3 b) => new(
      a.Y*b.Z - a.Z*b.Y, a.Z*b.X - a.X*b.Z,
      a.X*b.Y - a.Y*b.X);
}
```

*Operators must be **`static`**, and should be pure — one that mutates an operand will surprise every reader. **`Vec3`** here is a reference type, so each one is a heap object: fine for geometry and API boundaries, wasteful per element in a hot loop. For bulk arithmetic keep the data in **`Span<double>`** and use **`TensorPrimitives`**.*

Inheritance, Interfaces & Polymorphism

```
// INTERFACE = a contract, no state. Prefer it to a base class.
public interface IMaterial
{
  double Density { get; }
  void Stress(ReadOnlySpan<double> strain, Span<double> sigma);
  // Default body: optional to override, keeps the contract
     additive
  double Energy(ReadOnlySpan<double> strain) => double.NaN;
}

// ABSTRACT CLASS = shared state + partial behaviour. Cannot be
     new'd.
public abstract class Material(double rho) : IMaterial
{
  public double Density { get; } = rho;          // implements
     IMaterial
  public abstract void Stress(                   // MUST be
     overridden
      ReadOnlySpan<double> e, Span<double> s);
  public virtual double Energy(                  // MAY be
     overridden
      ReadOnlySpan<double> e) => double.NaN;
  public override string ToString() =>           // overrides object
      $"{GetType().Name}(rho={Density:G6})";
}

public sealed class Hooke(double rho, double E, double nu) :
     Material(rho)
{
  public override void Stress(ReadOnlySpan<double> e, Span<double>
      s)
  {
    double mu = E / (2 * (1 + nu)), tr = e[0] + e[1] + e[2];
    double lam = E * nu / ((1 + nu) * (1 - 2 * nu));
    for (int i = 0; i < 3; i++) s[i] = lam * tr + 2 * mu * e[i];
    for (int i = 3; i < 6; i++) s[i] = mu * e[i];
  }
  // base.X() calls the version this override replaced
  public override string ToString() => base.ToString() + $" E={E:
     G3}";
}

// Virtual dispatch: the RUNTIME type decides which body runs
IMaterial m = new Hooke(7800, 210e9, 0.3);
m.Stress(strain, sigma);                 // executes Hooke.Stress

// EXPLICIT implementation: reachable ONLY through the interface
public sealed class Rigid : IMaterial {
  public double Density => 0.0;
  void IMaterial.Stress(ReadOnlySpan<double> e, Span<double> s)
      => s.Clear();                      // not visible on Rigid
     itself
}
```

THE FOUR WORDS, IN ONE LINE EACH

`abstract`	no body here; every concrete child *must* supply one
`virtual`	there is a body; a child *may* replace it
`override`	this replaces the parent's body
`sealed override`	replaces it, and blocks any further override

TRAP: Omitting `override` *hides* instead of overriding. `class D : B { public double F() ... }` warns (CS0114), and `B b = new D(); b.F();` then runs `B.F`. Write `override`, or `new` if the hiding is deliberate.

TRAP: Virtual dispatch is an indirect call and blocks inlining. In an inner loop, call through the concrete `sealed` type rather than the interface — the JIT can devirtualise and inline that; it cannot inline through `IMaterial`.

One base class, any number of interfaces. Seal by default — leaving a class open promises its internals stay stable forever.

Generics, INumber⟨T⟩ & Constraints

```
// where T : class | notnull | unmanaged | new()
//       | IComparable<T> | INumber<T> | IFloatingPointIeee754<T>
//       | allows ref struct  -- lets T be Span<T> and friends

// INumberBase<T> static surface (numeric primitives expose):
//   T.Zero, T.One, T.AdditiveIdentity, T.MultiplicativeIdentity
//   T.Abs(x), T.Min(a,b), T.Max(a,b), T.Sign(x)
//   T.IsFinite(x), T.IsNaN(x), T.IsZero(x), T.IsNegative(x)
//   T.CreateChecked<U>(x)     -- throws OverflowException
//   T.CreateSaturating<U>(x) -- clamps to MinValue/MaxValue
//   T.CreateTruncating<U>(x) -- bit-truncates / wraps

// IFloatingPointIeee754<T> adds:
//   T.Pi, T.E, T.Tau, T.Epsilon, T.NaN, T.PositiveInfinity
//   T.Sqrt(x), T.Cbrt(x), T.Sin(x), T.Cos(x), T.Atan2(y,x)
//   T.Exp(x), T.Log(x), T.Pow(x,y), T.Hypot(x,y)
//   T.FusedMultiplyAdd(a,b,c)

public static T Norm<T>(ReadOnlySpan<T> x)
    where T : IFloatingPointIeee754<T>
{
  T s = T.Zero;
  for (int i = 0; i < x.Length; i++)
    s = T.FusedMultiplyAdd(x[i], x[i], s);  // fused per term
  return T.Sqrt(s);
}

// Generic class with ref-returning indexer
public sealed class DenseMatrix<T> where T : INumber<T>
{
  private readonly T[] _d;
  public int Rows { get; }
  public int Cols { get; }
  public DenseMatrix(int r, int c)
      { Rows = r; Cols = c; _d = new T[r * c]; }

  // ref T indexer: K[i,j] += dK is a real in-place update
  public ref T this[int i, int j] => ref _d[i * Cols + j];
  public Span<T> Row(int i) => _d.AsSpan(i * Cols, Cols);
  public Span<T> AsFlat() => _d.AsSpan();
}
```

Conversions, Casting & Boxing

```
// IMPLICIT: value-preserving, no cast written
int i = 42;  long l = i;  double d = i;       // widening

// EXPLICIT: information may be lost, so C# makes you say so
double x = 3.9;
int a = (int)x;                 //  3  TRUNCATES toward zero,
     never rounds
int b = (int)(-3.9);            // -3  toward zero, NOT toward -
     infinity
int c = (int)Math.Round(3.5);   //  4  banker's rounding
int e = (int)Math.Round(2.5);   //  2  ... yes, really
int f = Convert.ToInt32(3.9);   //  4  Convert ROUNDS; a cast
     TRUNCATES

// Integer division stays integer
double bad  = 1 / 2;            // 0.0   the classic first-day bug
double good = 1.0 / 2;          // 0.5

// Narrowing wraps silently unless you ask it not to
long big = 1L << 40;
int w = (int)big;               // 0, no diagnostic
int s = checked((int)big);      // OverflowException

// Generic math: name the out-of-range policy
T p = T.CreateChecked(x);       // throws outside range
T q = T.CreateSaturating(x);    // clamps to T.MinValue / T.
     MaxValue
T r = T.CreateTruncating(x);    // wraps (modular)

// Reference conversions
object o  = mesh;               // upcast: always safe
Mesh? m1  = o as Mesh;          // null if it is not a Mesh
if (o is Mesh m2) Use(m2);      // test + bind -- prefer this form
Mesh m3   = (Mesh)o;            // InvalidCastException if it is
     not

// USER-DEFINED: implicit only when it can neither fail nor
     surprise
public sealed record Metres(double Value) {
  public static implicit operator double(Metres m) => m.Value;
  public static explicit operator Metres(double v) => new(v);
}
double len = new Metres(2.5);   // implicit, silent
var  two   = (Metres)2.0;       // explicit, deliberate

// BOXING: a value type viewed through object or a non-generic
     interface
object boxed = 3.14;            // ALLOCATES on the heap, and
     copies
double back  = (double)boxed;   // unboxes (copies again)
IComparable ic = 5;             // ALLOCATES
```

TRAP: `(int)3.9` is 3 but `Convert.ToInt32(3.9)` is 4. Two spellings that look interchangeable give different answers; pick one convention per code base and write it down.

TRAP: Boxing is invisible in the source. It hides in `List<object>`, non-generic comparers, `string.Format` taking `object`, and any value type reached through an interface variable. Constrain a generic (`where T : INumber<T>`) and the boxing disappears.
as *works only for reference types and* ***T?****; everywhere else use* ***is T v****.* ***(T)x*** *on a reference type is a checked cast that throws — it is not a reinterpretation. To reinterpret* bits*, use* ***BitConverter*** *or* ***MemoryMarshal****.*

Extension Members & Lambdas

```
// CLASSIC extension methods (still works everywhere)
public static class VectorExt {
  public static double L2(this double[] v) {
    double s = 0;
    for (int i = 0; i < v.Length; i++) s += v[i]*v[i];
    return Math.Sqrt(s);
  }
}

// EXTENSION MEMBERS: properties, operators, statics
public static class SpanExt
{
  // Instance extension on a generic receiver named 'x'
  extension<T>(ReadOnlySpan<T> x) where T : INumber<T>
  {
    public bool IsEmpty => x.Length == 0;        // property
    public T Sum() {
      T s = T.Zero;
      for (int i = 0; i < x.Length; i++) s += x[i];
      return s;
    }
  }
  // Static extension: adds a static factory to Span<T>
  extension<T>(Span<T>) {
    public static Span<T> Of(T[] a) => a.AsSpan();
  }
  // Static extension on a numeric type
  extension(double) {
    public static double TwoPi => 2 * Math.PI;
  }
}
```

LAMBDAS, FUNC, ACTION

```
Func<double, double> f      = x => x * x;
Action<string>       print  = Console.WriteLine;
Predicate<double>    small  = x => Math.Abs(x) < 1e-12;
var inc = double (double x) => x + 1.0;       // typed

// Closures capture VARIABLES by reference, not values
double tol = 1e-10;
Predicate<double> tiny = x => Math.Abs(x) < tol;
tol = 1e-6; // 'tiny' now uses 1e-6
```

Enums & Flags

```
public enum BcKind   { Dirichlet, Neumann, Robin, Periodic }
public enum Solver : byte { CG = 1, GMRES = 2, DirectLU = 3 }

BcKind k = Enum.Parse<BcKind>("Neumann");
bool ok  = Enum.IsDefined<BcKind>(k);
foreach (BcKind v in Enum.GetValues<BcKind>()) { }

[Flags] public enum Dof
{
  None = 0,
  Ux = 1<<0, Uy = 1<<1, Uz = 1<<2,
  Rx = 1<<3, Ry = 1<<4, Rz = 1<<5,
  AllT = Ux | Uy | Uz,
  All  = AllT | Rx | Ry | Rz
}
Dof  pin    = Dof.Ux | Dof.Uy | Dof.Uz;
bool fixedX = (pin & Dof.Ux) != 0;       // hot-path style
pin |= Dof.Rz;                            // add a flag
pin &= ~Dof.Uy;                           // remove a flag
```

In hot paths use bitwise tests, not ***HasFlag*** *– the bitwise form is also clearer at a glance.*

Properties, Indexers & Events

```
public sealed class Sensor
{
  // Init-only auto-property
  public string Id { get; init; } = "";

  // Computed -- no backing field
  public double Magnitude => Math.Sqrt(X*X + Y*Y);

  // Field-backed validating property
  public double X {
    get;
    set => field = double.IsFinite(value)
      ? value : throw new ArgumentException("not finite");
  }
  public double Y { get; set; }

  // Indexer
  public double this[int axis] => axis switch {
    0 => X, 1 => Y,
    _ => throw new IndexOutOfRangeException()
  };

  // Event
  public event EventHandler<double>? Threshold;
  public void Update(double v) {
    X = v;
    if (Math.Abs(v) > 1e6) Threshold?.Invoke(this, v);
  }
}
```

Nullable Reference Types

```
#nullable enable

string  a = "ok";           // non-null by contract
string? b = MaybeText();    // may be null

if (b is { Length: > 0 })
  Console.WriteLine(b.Length);   // flow analysis proves non-null

// Prefer explicit nullable return values over side-effect outputs
Node? node = Lookup(id);
if (node is not null)
  Use(node);

// Null-forgiving operator: use only when you know better than
    flow analysis
Node root = possiblyNullRoot!;
```

Nullable reference types are compile-time analysis, not a runtime non-null guarantee. In new SDK projects ***<Nullable>enable</Nullable>*** *is normally on; keep it on for public numerical libraries.*

IDisposable, IAsyncDisposable, Lock

```
public sealed class NativeBuffer : IDisposable
{
  private unsafe void* _p;
  private readonly int _count;

  public NativeBuffer(int count) {
    if (count < 0) throw new ArgumentOutOfRangeException(nameof(
     count));
    _count = count;
    nuint bytes = checked((nuint)count * (nuint)sizeof(double));
    unsafe { _p = NativeMemory.AlignedAlloc(bytes, alignment: 64);
      }
  }

  public unsafe Span<double> AsSpan() => new((double*)_p, _count);

  public unsafe void Dispose() {
    if (_p is null) return;
    NativeMemory.AlignedFree(_p);
    _p = null;
  }   // sealed, no finalizer => no GC.SuppressFinalize needed
}
using var buf = new NativeBuffer(8192);

// Async cleanup
public sealed class AsyncWriter(Stream s) : IAsyncDisposable {
  private readonly Stream _stream = s;
  public async ValueTask DisposeAsync() {
    await _stream.FlushAsync();
    _stream.Dispose();
  }
}
await using var w = new AsyncWriter(File.Create("o.bin"));

// Modern lock primitive -- prefer over lock(object)
private readonly Lock _gate = new();
using (_gate.EnterScope()) { /* critical section */ }
```

If you write a finalizer, also call `GC.SuppressFinalize(this)` in `Dispose` to avoid the queue overhead when the user disposes deterministically.

COLLECTIONS

List, Dictionary, HashSet

```
// PRE-SIZE when count is known; resize copies are silent killers
var nodes = new List<int>(capacity: nNodes);
List<int> ids = [1, 2, 3];           // collection expr

nodes.Add(n);  nodes.AddRange(batch);
nodes.RemoveAll(x => x < 0);         // single-pass filter
nodes.EnsureCapacity(2 * nNodes);
Span<int> rawNodes = CollectionsMarshal.AsSpan(nodes); // no List
    resize while used

// Dictionary -- O(1) keyed lookup
var w = new Dictionary<int, double>(capacity: 256);
w[id] = value;                       // add or overwrite
w.TryAdd(id, value);                 // false on duplicate

// The BCL Try-pattern: bool result + 'out' for the value. You
    will
// READ this constantly, even if you never design an API around it
    .
if (w.TryGetValue(id, out double v))  // ONE lookup; v declared
    inline
    Use(v);                          // ... and still in scope
    here
double.TryParse(s, out double t);     // never throws; t = 0 on
    failure
w.Remove(id, out double gone);        // hands back what was
    removed
if (w.TryGetValue(id, out _)) { }     // '_' discards the value

// CollectionsMarshal.AsSpan(List<T>) is fast but invalidated by
    resize.
// For dictionaries, prefer ordinary APIs unless profiling demands
     otherwise.

// HashSet for repeated membership tests
var seen = new HashSet<int>(capacity: 1024);
bool isNew = seen.Add(id);           // false if duplicate
seen.UnionWith(other); seen.IntersectWith(other);

// Sorted (Red-Black tree; O(log n))
var sd = new SortedDictionary<int, string>();

// PriorityQueue: min-heap on TPriority
var pq = new PriorityQueue<string, double>();
pq.Enqueue("task", priority: 3.0);
string item = pq.Dequeue();              // removes minimum
    priority item
```

TRAP: Views returned by `CollectionsMarshal.AsSpan(List<T>)` are invalidated by any operation that may resize the list. Use such views only in tight, straight-line code where collection capacity is stable.

Async Streams & Channels

```
using System.Threading.Channels;

// Producer/consumer pipeline; bounded back-pressure
var ch = Channel.CreateBounded<double>(capacity: 1024);

_ = Task.Run(async () => {                  // producer
  for (int i = 0; i < n; i++)
    await ch.Writer.WriteAsync(i * h);
  ch.Writer.Complete();
});

await foreach (double v in ch.Reader.ReadAllAsync())
  Process(v);                               // consumer

// Async iterator method
public static async IAsyncEnumerable<string>
    StreamLinesAsync(string path)
{
  using var sr = new StreamReader(path);
  string? line;
  while ((line = await sr.ReadLineAsync()) is not null)
    yield return line;
}

// Lock-free MPMC queue; consumer APIs use the BCL Try-pattern.
// Hide that detail behind a small adapter if you dislike out
    parameters.
var q = new ConcurrentQueue<int>();
q.Enqueue(42);
```

Iterators, `yield` & Deferred Execution

```
// 'yield return' builds an IEnumerable<T> lazily: no list is
    materialised,
// and the method resumes where it left off on each MoveNext.
public static IEnumerable<int> BoundaryNodes(Mesh m) {
  for (int i = 0; i < m.Count; i++)
    if (m.IsBoundary(i)) yield return i;
}
foreach (int i in BoundaryNodes(mesh)) Assemble(i);

// 'yield break' ends the sequence early
public static IEnumerable<double> Residuals(double r0, double tol)
     {
  double r = r0;
  while (true) { yield return r; if (r < tol) yield break; r *=
    0.1; }
}

// DEFERRED: calling the method runs NOTHING. The body advances
    only
// while you enumerate -- so it sees the state at enumeration time
    .
var q = BoundaryNodes(mesh);    // no work happens on this line
mesh.Refine();                  // q will observe the REFINED mesh
int[] snap = BoundaryNodes(mesh).ToArray();   // force it now

// Enumerating twice re-runs the whole body twice.
foreach (var i in q) { }    // pass 1
foreach (var i in q) { }    // pass 2 -- recomputed from scratch
```

TRAP: Deferred execution plus mutation is a genuine source of wrong results: build the sequence, mutate the mesh, then enumerate, and you silently get the new mesh. If the caller may mutate, materialise with `ToArray()` at the boundary.
TRAP: An iterator method may not take `ref`/`out` parameters, and may not hold a `Span<T>` across a `yield`. Return counts and fill a caller-supplied `Span<T>` instead.
Every `foreach` over an `IEnumerable<T>` allocates an enumerator and dispatches each `MoveNext` virtually. Excellent for setup and traversal; in an inner kernel return a `ReadOnlySpan<T>` or an index range and loop with `for`.

LINQ: Useful, But Know the Cost

```
// Declarative and clear; often fine outside kernels
var top = nodes.Where(i => mass[i] > 0.0)
               .OrderBy(i => x[i])
               .Take(10)
               .ToArray();

// Hot-loop version: explicit, predictable, no iterator/delegate
    chain
int m = 0;
for (int k = 0; k < nodes.Count; k++) {
  int i = nodes[k];
  if (mass[i] > 0.0) scratch[m++] = i;
}
Array.Sort(scratch, 0, m, Comparer<int>.Create((i, j) => x[i].
    CompareTo(x[j])));
```

LINQ allocates iterator/delegate objects in many cases and can hide multiple passes. Use it freely for setup code; prefer explicit loops in low-level numerical kernels and parsers.

Span, Memory & stackalloc

```
// Span<T>: a stack-only view; zero-copy; synchronous only
// CANNOT: be a class field, be boxed, cross await.
double[] arr = new double[1024];
Span<double> row = arr.AsSpan(off, len);   // no allocation
Span<double> tmp = stackalloc double[16];  // stack buffer
tmp.Clear();

// Implicit conversions: T[] -> Span<T> at call sites
static double Dot(ReadOnlySpan<double> a, ReadOnlySpan<double> b)
{
  if (a.Length != b.Length) throw new ArgumentException();
  double s = 0;
  for (int i = 0; i < a.Length; i++) s += a[i] * b[i];
  return s;
}
double d = Dot(arr, arr);   // direct, no .AsSpan() needed

// scoped: prevents argument from escaping caller
static int CountPos(scoped ReadOnlySpan<double> x) {
  int c = 0;
  for (int i = 0; i < x.Length; i++) if (x[i] > 0) c++;
  return c;
}

// Memory<T>: heap-backed; survives async; can be a field
Memory<double> mem = new double[n];
await ProcessAsync(mem);              // Span<T> CANNOT do this
Span<double> s = mem.Span;            // extract inside sync code

// params Span<T>: variadic without array allocation
static double Max(params ReadOnlySpan<double> values) {
  double m = double.NegativeInfinity;
  foreach (double v in values) if (v > m) m = v;
  return m;
}
```

ref Returns & ref Locals

```
// ref return + ref local: writes flow back to the source array
static ref double At(double[] a, int i) => ref a[i];
ref double cell = ref At(arr, 10);
cell *= 2;                            // updates arr[10]

// ref readonly return: alias without copy, no mutation
static ref readonly Vec3 First(ReadOnlySpan<Vec3> x)
    => ref x[0];

// Prefer value-returning APIs over ref-parameter APIs.
static Vec3 Negated(Vec3 v) => new(-v.X, -v.Y, -v.Z);

static double ParseStrict(string s) =>
    double.Parse(s, CultureInfo.InvariantCulture);

// BCL Try* APIs use out; consume them when needed,
// but avoid designing your own numerical APIs around out
     parameters.
```

TRAP: Avoid `ref` parameters in public numerical APIs unless a measured interop or ABI constraint forces them. `ref` returns and short-lived `ref` locals are useful for indexed storage views; do not let such aliases outlive the storage operation that produced them.

ArrayPool, NativeMemory, GC Control

```
// Rent/return: avoids GC pressure for large transient buffers
var pool = ArrayPool<double>.Shared;
double[] buf = pool.Rent(n);   // may be larger than n!
try {
  Span<double> s = buf.AsSpan(0, n);
  s.Clear();                        // never assume zeroed
  Work(s);
}
finally { pool.Return(buf, clearArray: false); }

// Disposable wrapper for clean call-sites
public sealed class Scratch(int n) : IDisposable {
  private double[]? _b = ArrayPool<double>.Shared.Rent(n);
  public Span<double> Span => _b!.AsSpan(0, n);
  public void Dispose() {
    if (_b is null) return;
    ArrayPool<double>.Shared.Return(_b);
    _b = null;
  }
}
using (var s = new Scratch(4096)) Work(s.Span);

// Skip zero-init when all elements will be overwritten
double[] big = GC.AllocateUninitializedArray<double>(n);

// Pinned heap allocation -- never moves; safe for native libs
double[] pin = GC.AllocateArray<double>(n, pinned: true);

// Aligned native allocation (32 = AVX2; 64 = AVX-512)
unsafe {
  double* p = (double*)NativeMemory.AlignedAlloc(
      (nuint)(n * sizeof(double)), alignment: 64);
  try { /* use p[0..n-1] */ }
  finally { NativeMemory.AlignedFree(p); }
}
```

Arrays $>$ 85 kB land on the Large Object Heap (LOH), which is collected only on Gen 2. Use **`ArrayPool`** *or pinned allocation for these to avoid LOH fragmentation.*

MemoryMarshal, Pointers & Binary I/O

```
// Reinterpret a byte buffer as doubles (no copy)
Span<byte>   raw = stackalloc byte[64];
Span<double> dbl = MemoryMarshal.Cast<byte, double>(raw);

// Clear or fill through Span<T> first; the JIT usually removes
// redundant bounds checks in simple counted loops.
dbl.Clear();
for (int i = 0; i < dbl.Length; i++)
  dbl[i] = 0.0;

// Pin for raw pointer access in unsafe blocks
unsafe {
  fixed (double* p = dbl) { /* call native(p, ...) */ }
}

// Endian-aware binary I/O -- needs using System.Buffers.Binary;
Span<byte> bytes = stackalloc byte[8];
BinaryPrimitives.WriteDoubleLittleEndian(bytes, 3.14);
double back = BinaryPrimitives.ReadDoubleLittleEndian(bytes);

// Ref to first element; not pinned by itself
ref double headRef = ref MemoryMarshal.GetArrayDataReference(arr);
```

HOT-LOOP DISCIPLINE

- Capturing lambdas/delegates allocate or extend lifetimes; non-capturing delegates may be cached.
- Boxing via **`object`** or non-generic interfaces.
- **`string.Split`** on huge inputs; use **`SearchValues<char>`**.

CLOSURE CAPTURE TRAP

```
// BUG: every task captures the SAME 'i'
for (int i = 0; i < n; i++)
  tasks[i] = Task.Run(() => Work(i));   // all see final i

// FIX: introduce a fresh local
for (int i = 0; i < n; i++) {
  int j = i;
  tasks[i] = Task.Run(() => Work(j));
}
```

Native Interop: Keep Boundaries Explicit

```
using System.Runtime.InteropServices;

internal static partial class NativeKernels
{
  [LibraryImport("nativekernels", EntryPoint = "scale_inplace")]
  internal static unsafe partial void ScaleInPlace(
      double* data, int length, double factor);
}

// Keep managed arrays pinned only for the shortest possible scope.
unsafe {
  fixed (double* p = values) {
    NativeKernels.ScaleInPlace(p, values.Length, factor);
  }
}
```

For new interop code, source-generated ***`LibraryImport`*** *is usually preferable to reflection-based* ***`DllImport`****. Validate calling convention, array layout, ownership, and lifetime explicitly.*

SIMD & VECTORISED NUMERICS

TensorPrimitives — Use This First

```
// System.Numerics.Tensors: hardware-accelerated vector
// operations over Span<T>; auto-uses available SIMD where supported.
// NuGet/package may be needed depending on TFM.
using System.Numerics.Tensors;

ReadOnlySpan<double> a = xs, b = ys, c = zs;  // inputs
Span<double> y = dest;                        // len(y) == len(a)

// Element-wise: y = a + b   (NOT matmul!)
TensorPrimitives.Add(a, b, y);
TensorPrimitives.Subtract(a, b, y);
TensorPrimitives.Multiply(a, b, y);        // Hadamard product
TensorPrimitives.Divide(a, b, y);

// Scalar/broadcast and axpy-style operations
TensorPrimitives.Add(a, 1.0, y);           // y = a + 1
TensorPrimitives.Multiply(a, 2.0, y);      // y = 2*a
TensorPrimitives.MultiplyAdd(a, 2.0, b, y);// y = 2*a + b, not fused
TensorPrimitives.FusedMultiplyAdd(a, b, c, y); // y = fma(a,b,c)

// Reductions
var sum   = TensorPrimitives.Sum<double>(a);
var sumSq = TensorPrimitives.SumOfSquares<double>(a);
var norm2 = TensorPrimitives.Norm<double>(a);   // sqrt(sumSq)
var dot   = TensorPrimitives.Dot<double>(a, b);
var dist  = TensorPrimitives.Distance<double>(a, b);
var cos   = TensorPrimitives.CosineSimilarity<double>(a, b);
var mxV   = TensorPrimitives.Max<double>(a);
var mxI   = TensorPrimitives.IndexOfMax<double>(a);

// Math (vectorised where supported)
TensorPrimitives.Abs(a, y);   TensorPrimitives.Sqrt(a, y);
TensorPrimitives.Exp(a, y);   TensorPrimitives.Log(a, y);
TensorPrimitives.Sin(a, y);   TensorPrimitives.Cos(a, y);
TensorPrimitives.Sigmoid(a, y); TensorPrimitives.SoftMax(a, y);
```

Reach for ***`TensorPrimitives`*** *before hand-written SIMD. It handles tails and has tuned implementations.* ***`MultiplyAdd`*** *is the ordinary multiply-plus-add API; use* ***`FusedMultiplyAdd`*** *for one-rounding FMA semantics when available for the element type.*

Portable SIMD: Vector⟨T⟩

```
using System.Numerics;
// Vector<T> picks the widest hardware width at runtime
// (Vector128 / Vector256 / Vector512) -- count via Vector<T>.Count

static double DotSimd(ReadOnlySpan<double> a,
                      ReadOnlySpan<double> b)
{
  if (!Vector.IsHardwareAccelerated || a.Length < Vector<double>.Count)
    return DotScalar(a, b);

  int w = Vector<double>.Count;
  Vector<double> acc = Vector<double>.Zero;
  int i = 0;
  for (; i <= a.Length - w; i += w) {
    var va = new Vector<double>(a.Slice(i, w));
    var vb = new Vector<double>(b.Slice(i, w));
    acc += va * vb;
  }
  double s = Vector.Sum(acc);
  for (; i < a.Length; i++) s += a[i] * b[i];   // tail
  return s;
}
```

WIDTH-EXPLICIT (AVX-512 PATH)

```
using System.Runtime.Intrinsics;
// Vector256: 4 doubles; Vector512: 8 doubles (AVX-512)
if (Vector512.IsHardwareAccelerated) {
  Vector512<double> acc = Vector512<double>.Zero;
  int i = 0;
  for (; i <= a.Length - 8; i += 8) {
    var va = Vector512.Create<double>(a.Slice(i, 8));
    var vb = Vector512.Create<double>(b.Slice(i, 8));
    acc = Vector512.Add(acc, Vector512.Multiply(va, vb));
  }
  double s = Vector512.Sum(acc);
  for (; i < a.Length; i++) s += a[i] * b[i];  // required tail
}
```

Data layout and memory bandwidth often dominate instruction choice. Use ***`NativeMemory.AlignedAlloc`*** *when exact alignment matters; pinned managed arrays are non-moving but should not be treated as a portable 64-byte-alignment guarantee.*

Generic Math Kernels — Numerical Quality

```
// AXPY: y = alpha*x + y
public static void Axpy<T>(T alpha, ReadOnlySpan<T> x, Span<T> y)
    where T : INumber<T>
{
  if (x.Length != y.Length) throw new ArgumentException();
  for (int i = 0; i < x.Length; i++)
    y[i] += alpha * x[i];
}

// Kahan compensated sum: recovers the rounding lost each step
public static T SumKahan<T>(ReadOnlySpan<T> x)
    where T : IFloatingPointIeee754<T>
{
  T s = T.Zero, c = T.Zero;
  for (int i = 0; i < x.Length; i++) {
    T y = x[i] - c;
    T t = s + y;
    c = (t - s) - y;
    s = t;
  }
  return s;
}

// Neumaier variant: better when |x[i]| can exceed |sum|
public static T SumNeumaier<T>(ReadOnlySpan<T> x)
    where T : IFloatingPointIeee754<T>
{
  T s = T.Zero, c = T.Zero;
  for (int i = 0; i < x.Length; i++) {
    T t = s + x[i];
    c += T.Abs(s) >= T.Abs(x[i]) ? (s - t) + x[i] : (x[i] - t) + s
     ;
    s = t;
  }
  return s + c;
}

// Pairwise sum: simple, reproducible for fixed partitioning
public static T SumPairwise<T>(ReadOnlySpan<T> x) where T :
     INumber<T>
{
  if (x.Length <= 16) { T s = T.Zero; foreach (T v in x) s += v;
     return s; }
  int m = x.Length / 2;
  return SumPairwise(x[..m]) + SumPairwise(x[m..]); // span slices
     : no array
}

public static T Hypot<T>(T x, T y)
    where T : IFloatingPointIeee754<T> => T.Hypot(x, y);
```

TRAP: Generic math can inhibit some specialisation and inlining decisions. For small dense kernels over `double`, keep specialised overloads and profile against the generic version.

CONCURRENCY, ASYNC & PARALLEL

async / await, Task & ValueTask

```
// async returns Task / Task<T> / ValueTask / ValueTask<T>
public async Task<double[]> LoadAsync(string path,
    CancellationToken ct = default)
{
  string[] lines = await File.ReadAllLinesAsync(path, ct);
  return Array.ConvertAll(lines,
      s => double.Parse(s, CultureInfo.InvariantCulture));
}

double[] data = await LoadAsync("vec.txt");

// Run several awaits concurrently, then await the bundle
Task<double[]> t1 = LoadAsync("a.txt");
Task<double[]> t2 = LoadAsync("b.txt");
double[][] both = await Task.WhenAll(t1, t2);

// Cancellation: pass token everywhere; check periodically
var cts = new CancellationTokenSource();
cts.CancelAfter(TimeSpan.FromSeconds(30));
cts.Token.ThrowIfCancellationRequested();

// ValueTask: avoids heap alloc when synchronously cached
public ValueTask<double> CachedAsync(int key) =>
  _cache.TryGetValue(key, out double hit)   // ONE lookup
    ? ValueTask.FromResult(hit)
    : new ValueTask<double>(LoadFromDiskAsync(key));
```

TRAP: Do NOT `await` the same `ValueTask` twice. Convert to `Task` via `.AsTask()` if you need to.
TRAP: `Task.Run` is not a numerical parallel loop primitive. Use `Parallel.For/ForEach`, partitioned tasks, SIMD, or `TensorPrimitives` for CPU kernels; use `Task.Run` for UI/offload boundaries or unavoidable synchronous work.

Parallel & Thread Safety

```
// Data parallelism: each iteration MUST be independent
Parallel.For(0, n, i => results[i] = Compute(input[i]));

var opt = new ParallelOptions {
  MaxDegreeOfParallelism = Math.Max(1, Environment.ProcessorCount
     - 1),
  CancellationToken      = cts.Token
};

// Thread-safe reduction with thread-local accumulator
object sumLock = new();
double total = 0.0;
Parallel.For<double>(0, n,
  () => 0.0,
  (i, _, local) => local + values[i] * values[i],
  local => { lock (sumLock) total += local; });

// Deterministic partitioned reduction: no shared accumulator
int p = Environment.ProcessorCount;
double[] partial = new double[p];
Parallel.For(0, p, t => {
  int lo = (int)((long)t * n / p);        // widen, then narrow
  int hi = (int)((long)(t + 1) * n / p); // avoids t*n overflow
  double s = 0.0;
  for (int i = lo; i < hi; i++) s += values[i] * values[i];
  partial[t] = s;
});
double total2 = partial.Sum();

// volatile: visibility/order constraint, NOT compound atomicity
private volatile bool _running = true;
```

TRAP: For numerical reductions, prefer deterministic per-thread partials or a lock-protected final merge. Avoid shared floating-point accumulators in the loop body; their order is nondeterministic and usually slower.
TRAP: Beware false sharing: adjacent per-thread outputs can share a 64-byte cache line and ping-pong between cores. Partition by contiguous output slices or pad per-thread counters.

FILE I/O, STRINGS, REGEX & JSON

File & Stream I/O

```
using System.IO;

// One-shot
string text   = File.ReadAllText("data.txt");
string[] lns  = File.ReadAllLines("data.txt");
byte[] bytes  = File.ReadAllBytes("data.bin");
```

```
// Async (preferred for I/O)
text = await File.ReadAllTextAsync("data.txt");
await File.WriteAllLinesAsync("results.txt", lns);
```

```
// Streaming for large files
using var sr = new StreamReader("big.csv");
string? line;
```

```
while ((line = sr.ReadLine()) is not null) Process(line);

using var sw = new StreamWriter("log.txt", append: false);
sw.WriteLine($"# {DateTime.UtcNow:O}");

// Binary I/O: raw IEEE 754
using var bw = new BinaryWriter(File.Create("vec.bin"));
bw.Write(n);                              // int header
for (int i = 0; i < n; i++) bw.Write(v[i]);

// Path / Directory
string dir = Path.Combine("results", "run01");
Directory.CreateDirectory(dir);
bool ex    = File.Exists("data.txt");
string fp = Path.GetFullPath("rel/path");
```

For numerical data: prefer binary or text with `InvariantCulture`. Never let locale settings affect persisted numeric files.

Strings & Number Formatting

```
string s = $"x = {x:F6}, n = {n:D6}";       // interpolation
string p = @"C:\data\file.txt";             // verbatim
string j = """{"x":3.14}""";                // raw

// Numeric format specifiers, as used inside $"{v:__}"
//   G    shortest round-trippable    E6   1.234568E+003
//   G15  15 sig digits               F4   1.2346
//   G17  full double precision       D6   zero-padded 000042
//   R    legacy round-trip           X8   hex 0000002A

// ALWAYS use InvariantCulture for persisted numeric data
double v = double.Parse(token, CultureInfo.InvariantCulture);

// Span-based parse (zero allocation, throws on invalid input)
ReadOnlySpan<char> tok = line.AsSpan(start, len);
double x = double.Parse(tok, NumberStyles.Float,
    CultureInfo.InvariantCulture);

// Formatting for persisted data
string txt = v.ToString("G17", CultureInfo.InvariantCulture);
writer.Write(txt);

// Ordinal vs culture -- these are DIFFERENT operations
bool eq = a.Equals(b, StringComparison.Ordinal);       // code-
    unit-wise
bool ci = a.Equals(b, StringComparison.OrdinalIgnoreCase);
string.Compare("a", "B", StringComparison.Ordinal);          //
    +31 ('a'>'B')
string.Compare("a", "B", StringComparison.InvariantCulture); //
    -1 (a<B)

// Multi-character search at SIMD speed
SearchValues<char> delims = SearchValues.Create(",;\t ");
int pos = line.AsSpan().IndexOfAny(delims);  // .AsSpan() required

// StringBuilder: avoids O(n^2) concatenation
var sb = new StringBuilder(capacity: 1024);
sb.Append("Hello").AppendLine()
  .AppendFormat(CultureInfo.InvariantCulture, "{0:F4}", x);
```

`G17` for double is round-trip exact (`G9` for float). `R` is the historical round-trip specifier; use `G17` in new code.
TRAP: Culture-sensitive comparison depends on the machine's locale. Use `StringComparison.Ordinal` for keywords, file formats, dictionary keys, and anything a program parses; reserve culture-sensitive comparison for text shown to a person.

Regex — Source-Generated

```
using System.Text.RegularExpressions;

// CLASSIC: pattern compiled at first use
var rx = new Regex(@"^\s*(-?\d+(?:\.\d+)?)\s*$",
                   RegexOptions.Compiled);
Match m = rx.Match(input);
if (m.Success) {
  double v = double.Parse(m.Groups[1].ValueSpan,
      CultureInfo.InvariantCulture);
}

// MODERN: source-generated; zero startup cost, AOT-safe
public partial class Parsers
{
  [GeneratedRegex(
    @"^\s*(?<x>-?\d+(?:\.\d+)?)\s+(?<y>-?\d+(?:\.\d+)?)\s*$",
    RegexOptions.IgnoreCase)]
  public static partial Regex Coord();
}
Match m2 = Parsers.Coord().Match(line);
if (m2.Success) {
  double x = double.Parse(m2.Groups["x"].ValueSpan,
      CultureInfo.InvariantCulture);
  double y = double.Parse(m2.Groups["y"].ValueSpan,
      CultureInfo.InvariantCulture);
}

foreach (Match hit in Parsers.Coord().Matches(text)) { }
string clean = Regex.Replace(s, @"\s+", " ");
```

Use `Groups[i].ValueSpan` (not `Value`) to avoid allocating a new string per match – big win when parsing millions of lines.

System.Text.Json

```
using System.Text.Json;
using System.Text.Json.Serialization;

public sealed record Point(double X, double Y);
public sealed record Mesh(string Name,
                          IReadOnlyList<Point> Nodes);

string json = JsonSerializer.Serialize(mesh);
Mesh? back  = JsonSerializer.Deserialize<Mesh>(json);

// Options most relevant to scientific data
var opts = new JsonSerializerOptions {
  WriteIndented = true,
  PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
  PropertyNameCaseInsensitive = true,
  // For strict interchange, also consider newer unmapped/
     duplicate policies.
  // CRUCIAL: round-trip NaN, Infinity safely
  NumberHandling = JsonNumberHandling.
     AllowNamedFloatingPointLiterals,
};
JsonSerializer.Serialize(mesh, opts);

// Source-generated serializer
// AOT-safe, zero reflection, ~2x faster
[JsonSerializable(typeof(Mesh))]
[JsonSerializable(typeof(Point))]
[JsonSourceGenerationOptions(WriteIndented = true)]
public partial class MeshCtx : JsonSerializerContext { }

string s = JsonSerializer.Serialize(mesh,
    MeshCtx.Default.Mesh);

// Stream-based for large files
await using var fs = File.Create("results.json");
await JsonSerializer.SerializeAsync(fs, mesh);
```

Without `AllowNamedFloatingPointLiterals`, attempting to serialize `double.NaN` throws `JsonException`. Round-tripping FEM results almost always needs this.

Dates, Times, Random & Stopwatch

```
// PREFER DateTimeOffset over DateTime: explicit UTC offset
DateTimeOffset now = DateTimeOffset.UtcNow;
DateOnly d = DateOnly.FromDateTime(DateTime.Today);
TimeOnly t = TimeOnly.FromDateTime(DateTime.Now);

TimeSpan dt = now - start;
double secs = dt.TotalSeconds;

// ISO 8601 round-trip
string s = now.ToString("O", CultureInfo.InvariantCulture);
DateTimeOffset back = DateTimeOffset.Parse(s,
    CultureInfo.InvariantCulture);

// Stopwatch: high-res; NOT wall-clock
var sw = Stopwatch.StartNew();
DoWork();
sw.Stop();
double ms = sw.Elapsed.TotalMilliseconds;
```

```
// PORTABLE -- use TimeSpan, not raw ticks:
// Stopwatch.Frequency varies (1e7 on Win, ~1e9 on Linux);
// raw 'ticks' are not interchangeable with DateTime ticks.

// Random: thread-safe shared instance
double u = Random.Shared.NextDouble();   // [0, 1)
int    k = Random.Shared.Next(0, n);
// For reproducibility (e.g. Monte Carlo), use a SEEDED local
var rng = new Random(seed: 42);
// Crypto-strength: System.Security.Cryptography.
    RandomNumberGenerator
```

TRAP: `Random.Shared` uses xoshiro – excellent statistically but NOT cryptographic and NOT reproducible across machines/runs. Always seed a local `Random` for reproducible numerical experiments.

Attributes, Reflection & Source Gen

```
using System.Reflection;
using System.Diagnostics;

// CUSTOM ATTRIBUTE
[AttributeUsage(AttributeTargets.Property | AttributeTargets.Field
    )]
public sealed class UnitAttribute(string symbol) : Attribute
{
  public string Symbol { get; } = symbol;
}

public sealed class Reading {
  [Unit("m/s")] public double Velocity { get; init; }
  [Unit("Pa")]  public double Pressure { get; init; }
}

// REFLECTION: discover types, read attributes
foreach (PropertyInfo pi in typeof(Reading).GetProperties())
  if (pi.GetCustomAttribute<UnitAttribute>() is { } u)
    Console.WriteLine($"{pi.Name}: [{u.Symbol}]");

// Reflection invoke (slow; cache MethodInfo if hot)
MethodInfo mi = typeof(Math).GetMethod("Sqrt",
    [typeof(double)])!;
double r = (double)mi.Invoke(null, [4.0])!;   // 2.0

// SOURCE GENERATORS already in the BCL:
//   [GeneratedRegex]   regex without runtime compile
//   [JsonSerializable] reflection-free JSON
//   [LoggerMessage]    high-perf logging

// Activity: distributed-tracing spans (OpenTelemetry-compatible)
var src = new ActivitySource("MyApp");
using (var act = src.StartActivity("solve")) {
  act?.SetTag("n", n);
  Solve();
}
```

Diagnose live processes with `dotnet-counters monitor`, `dotnet-trace collect`, and `dotnet-dump analyze`. EventCounters expose GC stats, allocation rate, exceptions/sec.

TESTING, ASSERTIONS & REPRODUCIBILITY

Unit Tests for Numerical Code

```
// $ dotnet new xunit -n Solver.Tests
// $ dotnet add Solver.Tests reference Solver
// $ dotnet test                       // -c Release if timing
    matters
using Xunit;

public class NormTests
{
  [Fact]                                  // one fixed case
  public void Norm_OfUnitAxis_IsOne() =>
      Assert.Equal(1.0, Norm([1.0, 0.0, 0.0]), tolerance: 1e-15);

  [Theory]                                // table-driven: 1 row =
      1 test
  [InlineData(3.0, 4.0, 5.0)]
  [InlineData(0.0, 0.0, 0.0)]
  [InlineData(1e200, 1e200, 1.414213562373095e200)]  // must not
     overflow
  public void Hypot_IsOverflowSafe(double x, double y, double want
     ) =>
      Assert.Equal(want, double.Hypot(x, y),
                   tolerance: 1e-12 * Math.Abs(want));

  [Fact]
  public void Solve_RejectsSingular() =>
      Assert.Throws<ArithmeticException>(() => Solve(singular, rhs
     ));
}

// Compare with RELATIVE tolerance; a bare absolute tolerance is a
// unit trap -- it passes in metres and fails in millimetres.
static bool Close(double a, double b,
                  double rtol = 1e-12, double atol = 0.0) =>
    Math.Abs(a - b) <=
    Math.Max(atol, rtol * Math.Max(Math.Abs(a), Math.Abs(b)));
```

ASSERTIONS THAT COST NOTHING IN RELEASE

```
Debug.Assert(k.Rows == k.Cols, "stiffness must be square");

[Conditional("DEBUG")]      // CALL SITES vanish unless DEBUG is
    defined
static void CheckSymmetry(DenseMatrix<double> k) { /* O(n^2) check
     */ }

#if DEBUG
  Console.Error.WriteLine($"residual = {r:G17}");
#endif
```

TRAP: `Debug.Assert` disappears in Release. Use it for programmer errors you want caught during development; use a thrown exception for anything a *user* of the API can trigger.

What to Test in a Solver

- **Convergence rate, not one number.** Refine h and check the observed slope of $\log e$ against $\log h$. A bug that merely halves accuracy sails through any single-value test.
- **Invariants** are cheap and catch real defects: symmetry and positive-definiteness of $\mathbf{K}$, zero strain energy for every rigid-body mode, partition of unity of the shape functions, and the patch test.
- **Manufactured solutions.** Choose u, substitute to obtain the body force, then verify that the full pipeline recovers u. This is the only test that exercises assembly, boundary conditions, and the solver together.
- **Analytic special cases** pin absolute correctness where a closed form exists; convergence studies cover the rest.
- **Assert on the residual**, not on the iteration count — iteration counts change with any tolerance or preconditioner edit.
- **Seed every generator.** `new Random(42)`, never `Random.Shared`, in anything a test compares against.
- **Golden files:** write with `G17`, read back, and compare the *parsed doubles* within a tolerance. Never diff formatted numeric text.

REPRODUCIBILITY CHECKLIST

Pin `<LangVersion>` and the target framework; record the runtime version

(`Environment.Version`) and CPU in the output header; seed every RNG; avoid shared floating-point accumulators in parallel reductions, whose summation order varies run to run; and remember that different SIMD widths change the grouping of a reduction and therefore its last bits.
TRAP: A parallel reduction that passes today and fails on a wider machine is not flaky — it is order-dependent. Fix the partitioning (see *Parallel & Thread Safety*), do not loosen the tolerance until the test stops failing.

NUMERICAL BEST PRACTICES & REFERENCE

Version Scope

- Scope: C# 14 syntax and .NET 10 SDK/runtime/library surface relevant to numerical/scientific code.
- Some APIs in `System.Numerics.Tensors` may be package-provided or preview-versioned depending on the exact SDK/-package combination.
- Treat every performance rule as a hypothesis until BenchmarkDotNet or an application-level benchmark confirms it on the target CPU, OS, and runtime.
- For public libraries, keep nullable analysis enabled, document ownership of buffers, and separate safe APIs from unsafe fast paths.

Numerical Best Practices

- Default to `double`. Use `float` or `Half` only when memory bandwidth dominates and you have measured the win.
- `decimal` is base-10 and slow – use it for money, not science.
- Always pass `CultureInfo.InvariantCulture` when parsing or formatting persisted numeric data.
- Use compensated (Kahan or Neumaier) or pairwise summation for long sums; naive sum's error grows like $O(n \cdot \epsilon)$.
- Use `T.FusedMultiplyAdd(a, b, c)` or `Math.FusedMultiplyAdd` when one-rounding semantics improve robustness; measure throughput on target hardware.
- Reach for `TensorPrimitives` before writing manual SIMD: it handles tails and uses optimised implementations when available.
- Pre-size collections; `List<T>` resize copies are silent killers.
- Inside hot loops: indexed `for`, `ReadOnlySpan<T>` parameters, no closures, no `string.Split`.
- Pin allocations *outside* loops; rent from `ArrayPool` for large transients; use native aligned memory when exact SIMD alignment is required for interop or kernels.
- Use `checked{}` where overflow indicates a bug; `unchecked{}` only when wraparound is intended.
- Always pass `CancellationToken` through async chains.
- Validate inputs once at the public API boundary; trust the kernel.

MEASUREMENT DISCIPLINE

- Release build, no debugger, warm-up before timing.
- BenchmarkDotNet for microbenchmarks; `Stopwatch` for coarse.
- Track allocations as a first-class metric – `dotnet-counters`.
- Validate correctness BEFORE optimising. Fast wrong code is worthless.

Key Interfaces & Selection Rules

Interface	Use
`IEquatable<T>`	typed Equals; pair with `GetHashCode`
`IComparable<T>`	natural total order; enables `Sort`
`IComparer<T>`	external comparator
`IEqualityComparer<T>`	key equality for dict/hash
`IEnumerable<T>`	lazy iteration; supports `foreach`
`IReadOnlyList<T>`	indexed, immutable public APIs
`IList<T>`	only when caller must mutate
`IDisposable`	deterministic release; pair with `using`
`IAsyncDisposable`	async cleanup; `await using`
`IAsyncEnumerable<T>`	`await foreach` streaming
`INumberBase<T>`	arithmetic, `Create*` conversions
`INumber<T>`	adds comparison + sign
`IFloatingPointIeee754<T>`	adds Sqrt/Sin/Pi/NaN/FMA
`ISpanFormattable`	zero-alloc `TryFormat`
`ISpanParsable<T>`	zero-alloc `Parse` from span

SELECTION RULE

Accept the WEAKEST interface that satisfies the caller. For numerical kernels prefer `ReadOnlySpan<T>` or `Span<T>` over any interface – this bypasses virtual dispatch entirely and enables implicit `T[]` conversions.

INUMBERBASE CREATE METHODS

`T.CreateChecked<U>(x)` throws `OverflowException` outside range.
`T.CreateSaturating<U>(x)` clamps to `T.MinValue` / `T.MaxValue`.
`T.CreateTruncating<U>(x)` bit-truncates / wraps – use when you explicitly want modular arithmetic.

Numerical Correctness Traps

- `x == double.NaN` is always `false` – use `double.IsNaN(x)`.
- Sorting with NaN: `Array.Sort` uses `IComparable`, which puts NaN *first*. `<` returns `false` for any comparison with NaN. The two disagree.
- `0.1 + 0.2 != 0.3` – never compare floats with `==` for equality; use a tolerance: `Math.Abs(a - b) < tol * Math.Max(Math.Abs(a), Math.Abs(b))`.
- `Math.Round` uses *banker's rounding* by default (round-to-even). For school rounding pass `MidpointRounding.AwayFromZero`.
- Subtraction of nearly-equal numbers loses precision (*catastrophic cancellation*). Reformulate: $\sqrt{x^2+y^2}$ as `double.Hypot(x, y)`; $1-\cos x$ as $2\sin^2(x/2)$.
- Naive summation accumulates rounding error that can grow like $O(n\,\epsilon)$; pairwise summation reduces this to $O(\epsilon \log n)$ and compensated (Kahan/Neumaier) schemes to roughly $O(\epsilon)$. The constants depend on conditioning.
- `Math.FusedMultiplyAdd(a, b, c)` is specified as one rounding; `a * b + c` is normally rounded after multiply and again after add.
- Integer overflow is silent by default. Wrap risky arithmetic in `checked{}` or compile with `<CheckForOverflowUnderflow>`.
- `double` keys in `Dictionary`: `double.NaN.Equals(double.NaN)` is `true`, while `double.NaN == double.NaN` is `false`. Avoid floating-point keys unless you define the exact equality semantics.
- Persisted `double` round-trips need `G17`, not `G15`. `float` needs `G9`.